รายงานโครงงาน

Void Gazer

การพัฒนาเกมยานอวกาศสองมิติ

ด้วยภาษา **Python** และ **Pygame**

อาจารย์ที่ปรึกษา ดร.สรายุทธ กรวิรัตน์



ผู้จัดทำโครงงาน

รายงานโดย

1. ปิยะพงศ์ อินทะจินดา

ชั้น CE6921 รหัสนักศึกษา 691540006002-9

2. ณัฐพงศ์ อนาวงศ์

ชั้น CE6921 รหัสนักศึกษา 691540006005-2

VOID GAZER
Gameplay Systems

วงจรการทำงานของเกม และการเปลี่ยนสถานะ

1. **Receive Event**
   รับเหตุการณ์จากผู้เล่นและระบบ
2. **Check Status**
   ตรวจสอบสถานะปัจจุบันของเกม
3. **Adjust Players, Enemies, Ammo, and Time**
   ปรับปรุงผู้เล่น, ศัตรู, กระสุน และ เวลา
4. **Check for Collisions**
   ตรวจสอบการชนกัน
5. **Update Scores and Boss Conditions** (Death or Win)
6. **Draw and Display on Screen**
   วาดและแสดงผลบนหน้าจอ

ฟังก์ชัน Pause

จำลองการทำงานของระบบสำคัญอย่างต่อเนื่อง เพื่อความสอดคล้อง

- **เวลา** (Time)
  จำลองเวลาให้เดินต่อเนื่อง
- **ตำแหน่ง** (Position)
  อัปเดตตำแหน่งของผู้เล่น
- **ไนโตร** (Nitro)
  ฟื้นฟูหรือไนโตรตามเวลา
- **การเตือน** (Alerts)
  ตรวจจับและเตรียมแจ้งเตือน
- **เสียง** (Sounds)
  เส้นเสียงและ SFX อย่างต่อเนื่อง

เพื่อให้เกมเมื่อกลับมาเล่นได้อย่างสอดคล้องและยุติธรรม



สถานะของเกม (Game States)

- **เมนู** (Menu)
  จุดเริ่มต้นและหน้าเมนูของเกม



- **กำลังเล่น** (Currently Playing)
  เกมกำลังดำเนินการ



- **พัก** (Pause)
  หยุดชั่วคราว



- **ชนะ** (Win)
  ผ่านด่านหรือเอาชนะบอส
- **จบเกม** (Game Over)
  ผู้เล่นพ้ายแพ้

การควบคุมและการเคลื่อนที่

Player Controls and Movement

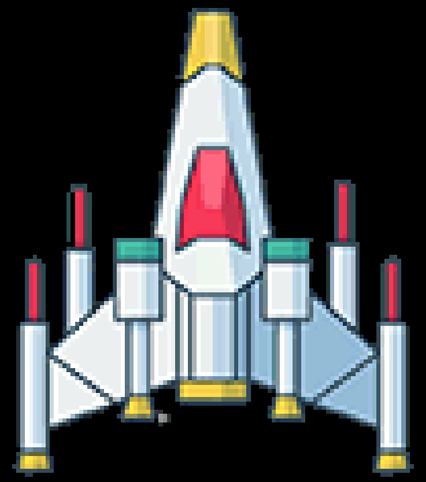
ทิศทางการเคลื่อนที่	ทิศทางการเล็ง
	
คีย์บอร์ด กำหนดการเคลื่อนที่	เมาส์กำหนดทิศทางการเล็ง อิสระจากการเคลื่อนที่



แยกทิศทางการเคลื่อนที่ออกจากการเล็ง

ยานจะเคลื่อนที่ไปในทิศทางที่ผู้เล่นสิ่ง
ขณะที่การเล็งอาวุธอิสระตามตำแหน่งเมาส์
ช่วยให้ควบคุมการต่อสู้ได้แม่นยำและยืดหยุ่น





การคำนวณการเคลื่อนที่

New **Position** = Original Position + **Speed** × dt

ความเร็วของยาน

ความเร็วปกติ	400	พิกเซล / วินาที
ความเร็วไนโตร	820	พิกเซล / วินาที

ระบบไนโตร

ความจุ	100	
อัตราการใช้	40	หน่วย / วินาที
อัตราการฟื้นฟู	25	หน่วย / วินาที



ศัตรูและบทบาทในเกม

แต่ละศัตรูมีบทบาทและรูปแบบการโจมตีที่แตกต่างกันทำลายศัตรูเพื่อสะสมคะแนนและอยู่รอดให้นานที่สุด

YELLOW LV 1



100 POINTS

2 HITS

Predictive triple charge

GREEN LV 3



300 POINTS

3 HITS

4-shot burst + flank shots

RED LV 5



600 POINTS

5 HITS

Strafes + 3 spread volleys

UFO BONUS



1,000 POINTS

2 HITS

Faster evasion; escapes 8s

สัญญาณเตือนเมื่อเกิดอันตราย

กำหนดพื้นที่การต่อสู้และแจ้งเตือนอันตรายให้ผู้เล่นรับรู้ล่วงหน้า
เพื่อสร้างการตัดสินใจที่แม่นยำและลดความสุญเสียโดยไม่จำเป็น

BOSS FIELD RADIUS

ชนาดพื้นที่เริ่มต้นและการปรับตามระยะห่างของบอส

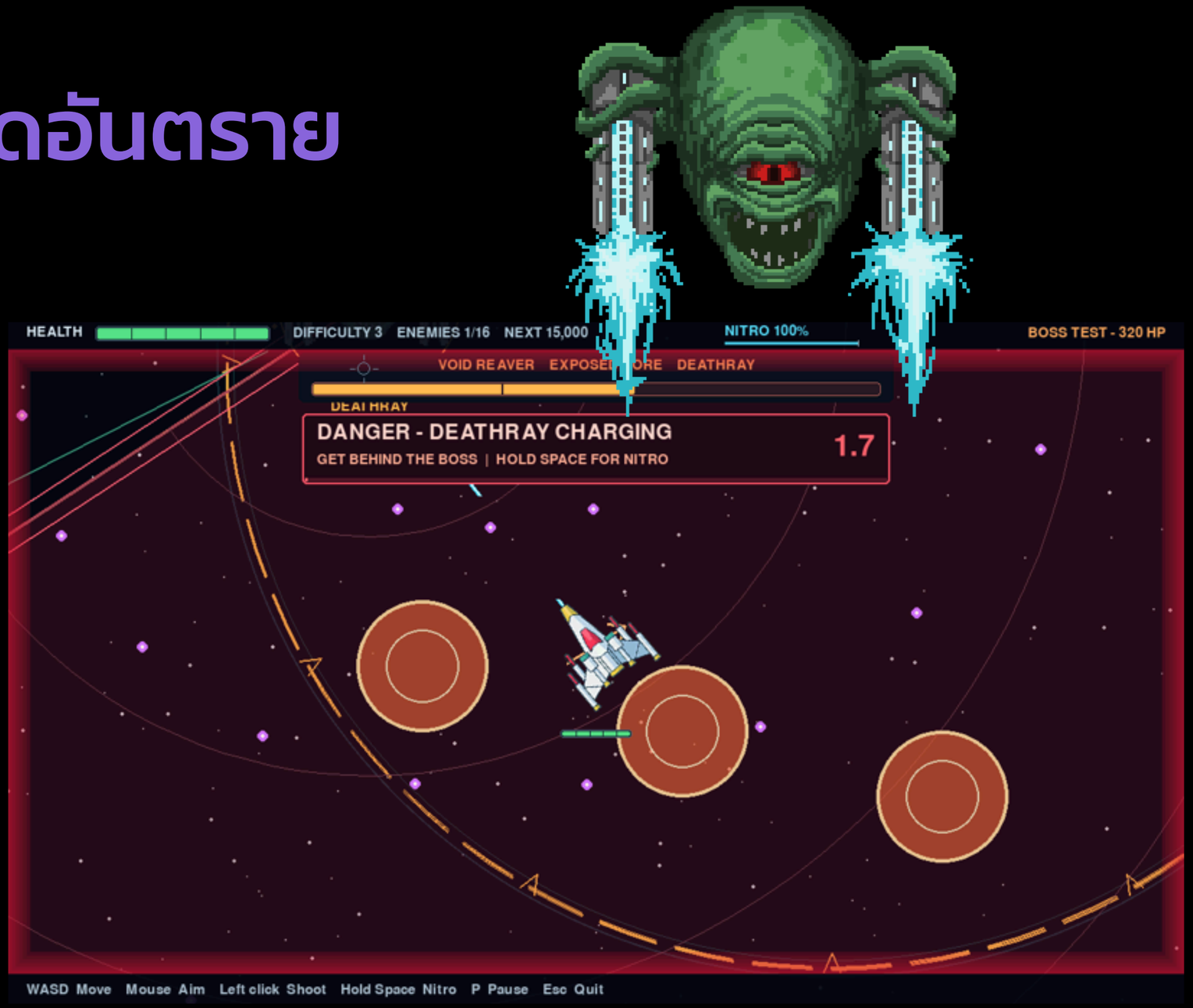
ระยะใกล้
เริ่มต้น
720

ระยะปานกลาง
บอสห่างออกไป
660

ระยะโกล
บอสอยู่ใกลมาก
600

0.85

หากอยู่นอกพื้นที่
เกินเวลาที่กำหนด
ผู้เล่นจะได้รับความ
เสียหาย



Void Gazer

ขั้นตอนการทำงานและโครงสร้างโปรแกรม

Requirements Analysis
วิเคราะห์ความต้องการและขอบเขตของเกม

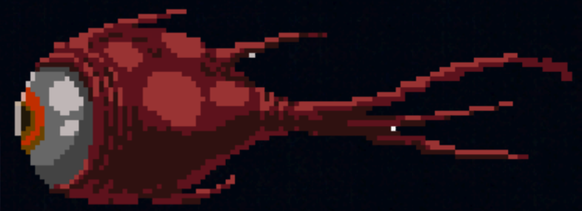
Data Design and File Separation
ออกแบบข้อมูลและแยกไฟล์ตามหน้าที่

Basic System Development
พัฒนาระบบพื้นฐานของเกม เช่น ผู้เล่น การเคลื่อนที่ กระสุน

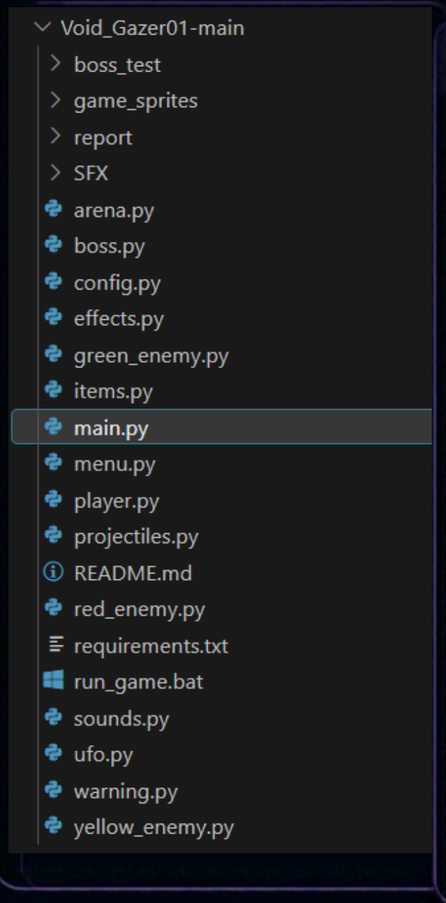
Adding Bosses and Complex Systems
เพิ่มบอสและระบบที่ซับซ้อน เช่น แพทเทิร์นการโจมตี

Scope Testing
ทดสอบระบบในขอบเขตที่กำหนด

Debugging and Retesting
แก้ไขข้อผิดพลาดและทดสอบซ้ำในระบบให้เสถียร



โครงสร้างโปรเจกต์ (ตัวอย่างไฟล์หลัก)



id_Gazer01-main > main.py > ...

```python
import argparse
import math
import random
import pygame

import config
from arena import update_arena, draw_arena, draw_arena_warning
from boss import Boss, preload as preload_boss
from effects import Explosion, preload
from items import maybe_drop, update_items, draw_items
from menu import Menu
from player import Player
from projectiles import update_enemy_bullets, draw_enemy_bullets
from sounds import BossSounds, EnemyDeathSounds, PlayerDeathSound, Dang
from warning import DangerDisplay
from yellow_enemy import YellowEnemy
from green_enemy import GreenEnemy
from red_enemy import RedEnemy
from ufo import UFO


def create_stars():
    stars = []
    for number in range(config.STAR_COUNT):
        depth = random.uniform(0.2, 0.8)
        brightness = int(80 + depth * 180)
        stars.append({
            "x": random.randrange(config.WIDTH),
            "y": random.randrange(config.HEIGHT),
            "depth": depth,
            "color": (brightness - 15, brightness - 5, brightness),
            "size": 2 if depth > 0.6 else 1,
        })
    return stars
```

แนวคิดสำคัญ

แยกหน้าที่ของแต่ละโมดูลให้ชัดเจน เพื่อให้ง่ายต่อการพัฒนา ทดสอบ และบำรุงรักษา พร้อมรองรับการขยายระบบในอนาคต

ภาพรวมโครงงาน

Void Gazer

สร้างเกมยานอวกาศ2Dด้วย**Python**และ**Pygame**

ผู้เล่นควบคุมยานอวกาศผ่านด่านต่างๆ
และ**ต่อสู้กับศัตรู**ในอวกาศ

เสริมทักษะการ**เขียนโปรแกรม**
และการ**ออกแบบเกม**อย่างเป็นระบบ

บูรณาการ**ความคิดสร้างสรรค์**
สู่ประสบการณ์การเล่นที่น่าตื่นเต้น



เทคโนโลยีและ
เครื่องมือที่ใช้

ภาษาโปรแกรม **Python**

ไลบรารีเกม **Pygame**

เครื่องมือช่วยพัฒนา

โปรแกรมแก้ไขโค้ด (IDE) เช่น **VSCode** หรือ **PyCharm**

ระบบทดสอบและ**ดีบัก**



ฟีเจอร์หลักของเกม

Void Gazer

ระบบควบคุมยานอวกาศแบบ **2 มิติ**

ศัตรูและการ**ยิงตอบโต้**

ด่านและระดับ**ความยากเพิ่มขึ้น**

ระบบเก็บ**คะแนน**และ**พลังชีวิต**



กระบวนการพัฒนาและความท้าทาย

01 ออกแบบ

02 เขียนโค้ด

03 ทดสอบ

04 แก้ไขบั๊ก

ความท้าทายที่พบ

การจัดการการชนทำให้เกิดพฤติกรรมไม่คาดคิด
การควบคุมยานไม่ลื่นไหลและไม่แม่นยำ

แนวทางแก้ไข

ปรับปรุงโค้ดและกระบวนการตรวจจับการชน
ทดสอบซ้ำเพื่อเพิ่มความเสถียรและความแม่นยำ



วัตถุประสงค์และขอบเขต

วัตถุประสงค์

พัฒนาเกม 2D แนวอวกาศ
แบบเล่นคนเดียวบน PC
ด้วยภาษา Python และ Pygame

คอนเซปต์ของโครงงาน

- พัฒนาเกมสำหรับผู้เล่นคนเดียว (Single Player)
- แพลตฟอร์ม: PC
- ควบคุมและระบบหลักตามรายละเอียดที่กำหนด
- ความละเอียดหน้าต่างและอัตราเฟรมตามที่กำหนด
- ระบบคะแนน ชีวิต และบอสตามเงื่อนไขที่กำหนด

การควบคุม

W A S D	เคลื่อนที่ยาน
เมาส์	เล็งเป้า
คลิกซ้าย	ยิง
Space	ใช้โนโตร
P	หยุดเกมชั่วคราว



ความสะเอียดหน้าต่าง	อัซเาฟรนเข้าหวย	อีวิตรึ่งต้นซองลู่แน่	บอลปรากฏแม็งคะแนนใจ	พลังบีวิตบอส
1280×800 พักเยล	60 FPS F9 -	5 แต้ม	10,000 คะแนน	320 แต้ม